

Capstone Project

Multi-Stage AI Workflow

AI-Powered API Test Suite Generator

Specialization: QA Engineering
Course: Applied AI & Prompt Engineering
Date: February 17, 2026
Presented by: Tob Adoba

1. Problem Statement

Manual API test case creation in QA is a slow, repetitive, and error-prone process. A QA engineer working on a typical REST API project must:

- Manually read and interpret API documentation endpoint by endpoint
- Identify positive, negative, and edge case scenarios from scratch
- Write boilerplate test code repeatedly for each endpoint
- Ensure consistent coverage across all HTTP methods (GET, POST, PUT, PATCH, DELETE)

This process typically takes 4 to 6 hours for a single API and often results in incomplete coverage, particularly for negative and edge cases, due to time constraints and human oversight.

This project proposes and demonstrates an AI-powered workflow that reduces this effort to under 45 minutes while producing higher-quality, more comprehensive test coverage by chaining two different AI UX types: a Chat AI and an IDE AI.

2. Workflow Overview

The workflow chains two AI tools across distinct UX types, where the output of the first tool becomes the direct input for the second:

Chat AI (Claude) → IDE AI (GitHub Copilot Chat) → Working Test Suite

Target API: JSONPlaceholder (<https://jsonplaceholder.typicode.com>)

Test Framework: REST Assured + JUnit 5 (Java Maven)

Stage 1 Tool: Claude (Chat AI) - Test scenario generation

Stage 2 Tool: GitHub Copilot Chat in VS Code (IDE AI) - Code generation

3. Step-by-Step Workflow Instructions

Phase 1: Chat AI - Generate Test Scenarios (Claude)

The first stage uses Claude as a Chat AI to analyze the JSONPlaceholder API and generate structured, comprehensive test scenarios. Claude acts as a QA analyst, producing a test plan document that covers all required test types.

Input: JSONPlaceholder API description (base URL, endpoints, HTTP methods supported)

Output: test-scenarios.md - a structured markdown file with 15 test scenarios across 3 categories

The scenarios generated covered:

- 7 Positive test cases (successful operations across all HTTP methods)
- 4 Negative test cases (error handling, invalid inputs, unsupported methods)
- 4 Edge cases (boundary values, special characters, large payloads, invalid params)

Phase 2: IDE AI - Generate Test Code (GitHub Copilot Chat)

The second stage feeds the test scenarios document directly into GitHub Copilot Chat inside VS Code. The IDE AI uses the structured scenarios as a specification to generate complete, production-ready Java test code.

Input: test-scenarios.md (output from Stage 1) + Maven project structure

Output: 3 complete Java files: BaseTest.java, Post.java, PostsAPITest.java

Copilot Chat generated the following files:

- BaseTest.java - REST Assured base configuration with @BeforeAll setup
- Post.java - POJO model class representing the API response structure
- PostsAPITest.java - 15 complete test methods using given/when/then pattern

Phase 3: Test Execution & Verification

The generated test suite was executed using Maven from the terminal to verify end-to-end functionality:

```
mvn clean test
```

All tests executed successfully against the live JSONPlaceholder API, confirming the workflow produced functional, working test code without manual corrections.

4. Prompts Used at Each Stage

Stage 1 Prompt - Claude (Chat AI)

The following prompt was used to generate test scenarios from Claude:

```
You are a QA engineer. Analyze the JSONPlaceholder REST API at
https://jsonplaceholder.typicode.com and generate comprehensive test
scenarios for the /posts endpoint. Include:
- 7 positive test cases covering all HTTP methods
- 4 negative test cases covering error handling and invalid inputs
- 4 edge cases covering boundary values, special characters,
  large payloads and invalid query parameters
Format the output as a structured markdown document with scenario name,
endpoint, method, request body (if applicable), and expected result.
```

Stage 2 Prompt - GitHub Copilot Chat (IDE AI)

The following prompt was fed into GitHub Copilot Chat in VS Code, using the test-scenarios.md as context:

```
I need you to generate a complete REST Assured test suite for the
JSONPlaceholder API based on my test scenarios document.

CONTEXT:
- Project: Maven Java project with REST Assured + JUnit 5
- Target API: https://jsonplaceholder.typicode.com
- Test scenarios are in test-scenarios.md
- Packages already exist: config, models, tests, utils

REQUIREMENTS:
1. CREATE BaseTest.java in src/test/java/com/qa/capstone/config/
  - Set base URI, configure logging, use @BeforeAll

2. CREATE Post.java in src/test/java/com/qa/capstone/models/
  - POJO with: userId, id, title, body fields
  - Include getters, setters, constructors, toString
```

```
3. CREATE PostsAPITest.java in src/test/java/com/qa/capstone/tests/  
- Extend BaseTest  
- Implement all 15 test scenarios using given/when/then pattern  
- Use Hamcrest matchers for assertions  
- Use JUnit 5 @Test annotations
```

```
Generate all files with complete, working Java code.
```

5. Results & Outcome

The AI-powered workflow successfully produced a complete, working API test suite. The key outcomes were:

- 15 test cases generated and executed covering all HTTP methods
- 3 test categories covered: positive, negative, and edge cases
- 3 Java source files generated with no manual code writing required
- Total workflow time: approximately 40 minutes (vs 4-6 hours manually)
- Test execution: all tests ran successfully against the live API

The data handoff between tools was clean and effective. Claude's markdown output served as a precise specification that Copilot Chat could interpret and implement directly, demonstrating the power of chaining AI tools with structured intermediate outputs.

6. Evaluation Rubric Mapping

Criteria	Weight	How This Project Meets It
Functionality	40%	15 REST Assured tests run end-to-end via Maven. All tests execute and produce verifiable pass/fail results against the live JSONPlaceholder API.
Efficiency	20%	Manual test case creation estimated at 4-6 hours. AI-powered workflow reduced this to under 45 minutes, with higher coverage (15 scenarios vs typical 5-7 manual tests).
Documentation	20%	Clear workflow diagram, step-by-step documentation, and all prompts recorded at each AI stage.
Adaptability	20%	Workflow logic is tool-agnostic. Any Chat AI + any IDE AI with REST Assured support can replicate this process. Not locked to specific proprietary tools.

7. Tools Used

Chat AI (Stage 1): Claude by Anthropic (claude.ai) - Used for API analysis and test scenario generation

IDE AI (Stage 2): GitHub Copilot Chat in VS Code - Used for complete test code generation

Build Tool: Apache Maven 3.x

Test Framework: REST Assured 5.3.2 + JUnit 5.10.0

Language: Java 11

Target API: JSONPlaceholder (<https://jsonplaceholder.typicode.com>)
